

NAME K.D.D. Sai AbhisamSUBJECT Deep learning lab obde

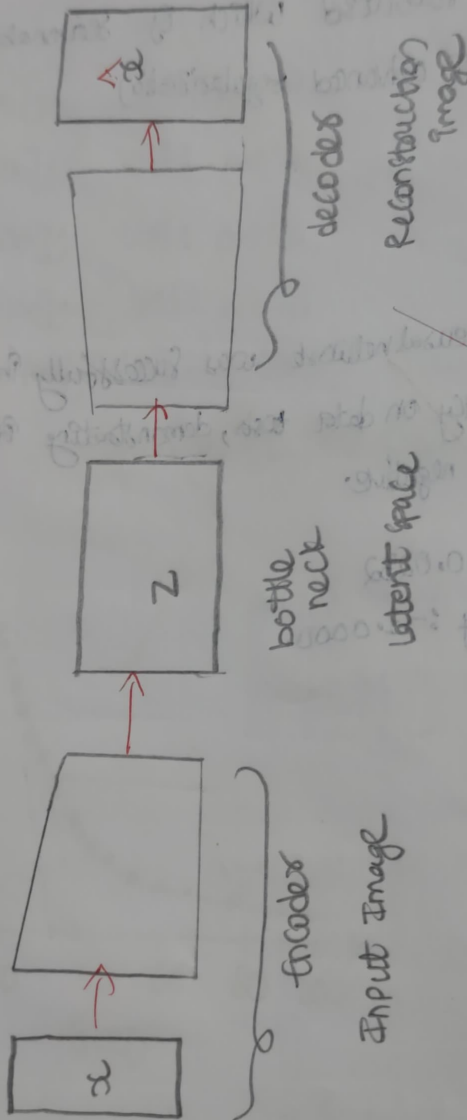
DIV. _____ ROLL NO. _____ SCHOOL _____

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
1		Analyze and exploring the deep learning Platforms	24-07-23	He
2		Implement a classifier using open-source data set	7/8/23	He 11/11/23
3		study of the classifiers with respect to statistical parameters	7/8/23	He 11/11/23
4		Build a simple feed forward neural network to recognize hand written characters	14/8/23	He 11/14/23
5		Study of activation function and its role	28-8-23	He 11/9/23
6		Implement gradient descent & back propagation in deep neural network.	09-9-23	He 11/9/23
7		Build a CNN model to classify Cats & dogs Images	16-9-23	He 11/23/23
8		Experiment using LSTM		He 11/12/23
9		Build a RNN		
10		Perform Compression on MNIST data set using Autoencoder		He 11/12/23
11		to implement experiment using Variational Autoencoder		
12		Implement DCGAN to generate Colour images	27/10	He 11/27/23
13		understanding the architecture of Pre-trained model	27/10	
14		Pre-trained CNN model as feature extractor using TLM		
15		Implement a YOLO model to detect objects		

samples ~~He~~

AE

Auto encoder architecture



10. Perform Compression on MNIST Dataset using Autoencoders

Aim: to design and implement an Autoencoder using PyTorch to compress and reconstruct images from the MNIST handwritten digit dataset.

Objective:

- ① to understand the concept of autoencoders as an unsupervised learning model for data compression and reconstruction.
- ② to apply autoencoders for dimensionality reduction
- ③ to visualize how well the compressed representation
- ④ to analyze the performance of the autoencoder based on quality.

Pseudo Code

- ① Import necessary libraries (torch)
- ② Load MNIST dataset
normalize images and convert them to tensor
- ③ define Autoencoder
encoder :- image ($28 \times 28 \rightarrow 3$) features
decoder :- reconstruct image ($3 \rightarrow 28 \times 28$)
- ④ Initialize
loss function = mean squared error
optimizer = Adam
- ⑤ for each epoch
flatten the image ($28 \times 28 \rightarrow 784$)
forward pass through encoder and decoder

- ⑥ test model on test data
- ⑦ visualize on test data
- ⑧ END

Observations

- * Performs real time detection
- * Algorithm used for high accuracy
- * It is ^{very} speed to detect.

Result

Detected objects are accurately localized and labeled on test images.

~~1/1/2023~~
~~1/1/2023~~
~~1/1/2023~~

[1]

✓ 25

```
# Get a batch of images from the test_loader
dataiter = iter(test_loader)
images, labels = next(dataiter)
images = images.to(device)

# Pass the original images through the trained model to get the reconstructed images
reconstructed_images = model(images)

# Select a subset of original and reconstructed images to visualize (e.g., the first 10)
n = 10
original_images_subset = images[:n].cpu().detach()
reconstructed_images_subset = reconstructed_images[:n].cpu().detach()

# Create a figure and a set of subplots to display the original and reconstructed images side-by-side
fig, axes = plt.subplots(2, n, figsize=(20, 4))

# Iterate through the selected images and display each original image in one subplot and its corresponding reconstructed image in an adjacent subplot
for i in range(n):
    # Display original image
    axes[0, i].imshow(original_images_subset[i].squeeze(), cmap='gray')
    axes[0, i].set_title("Original")
    axes[0, i].axis('off')

    # Display reconstructed image
    axes[1, i].imshow(reconstructed_images_subset[i].squeeze(), cmap='gray')
    axes[1, i].set_title("Reconstructed")
    axes[1, i].axis('off')

# Display the plot
plt.tight_layout()
plt.show()
```

[6]

✓ Os



```
# 1. Select a small sample of images from the test_loader.
# We can reuse the images and reconstructed images from the previous visualization step.
# Let's use the first 10 images from the previous batch.
sample_original_images = original_images_subset
sample_reconstructed_images = reconstructed_images_subset

# 2. Pass the selected original images through the encoder part of the trained model to obtain the compressed representation (latent vectors).
# Ensure the input is flattened as expected by the encoder
sample_original_images_flat = sample_original_images.view(-1, 28 * 28).to(device)
sample_compressed_representation = model.encoder(sample_original_images_flat)

# 3. Pass the obtained compressed representations through the decoder part of the trained model to reconstruct the images.
# This step was already done in the previous visualization subtask to get sample_reconstructed_images.
# To be explicit and follow the instructions, let's re-calculate it using the compressed representation.
sample_reconstructed_images_from_compressed = model.decoder(sample_compressed_representation).view(-1, 1, 28, 28).cpu().detach()

# 4. Store the original images, compressed representations, and reconstructed images for further analysis.
# The variables are already created in the previous steps:
# sample_original_images (on CPU, detached)
# sample_compressed_representation (on device, needs to be moved to CPU and detached)
# sample_reconstructed_images (on CPU, detached) - let's use the one calculated explicitly from compressed representation
sample_compressed_representation = sample_compressed_representation.cpu().detach()
sample_reconstructed_images = sample_reconstructed_images_from_compressed

print("Sample original images shape:", sample_original_images.shape)
print("Sample compressed representation shape:", sample_compressed_representation.shape)
print("Sample reconstructed images shape:", sample_reconstructed_images.shape)
```



```
Sample original images shape: torch.Size([10, 1, 28, 28])
Sample compressed representation shape: torch.Size([10, 3])
Sample reconstructed images shape: torch.Size([10, 1, 28, 28])
```



```

# 4. Loss and optimizer
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)

# 5. Training parameters
num_epochs = 20

# 6. Training the model
print("Starting training...")
for epoch in range(num_epochs):
    for data in train_loader:
        images, _ = data
        images = images.to(device)

        # Forward pass
        reconstructed_images = model(images)
        loss = criterion(reconstructed_images, images)

        # Backward and optimize
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}')

print("Training finished.")

```

```

Starting training...
Epoch [1/20], Loss: 0.1902
Epoch [2/20], Loss: 0.1591
Epoch [3/20], Loss: 0.1591
Epoch [4/20], Loss: 0.1545
Epoch [5/20], Loss: 0.1491
Epoch [6/20], Loss: 0.1690
Epoch [7/20], Loss: 0.1550
Epoch [8/20], Loss: 0.1242
Epoch [9/20], Loss: 0.1422
Epoch [10/20], Loss: 0.1192
Epoch [11/20], Loss: 0.1466
Epoch [12/20], Loss: 0.1243
Epoch [13/20], Loss: 0.1389
Epoch [14/20], Loss: 0.1348

```

```

self.encoder = nn.Sequential(
    nn.Linear(28 * 28, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 12),
    nn.ReLU(),
    nn.Linear(12, 3) # Latent space dimension
)

# Decoder
self.decoder = nn.Sequential(
    nn.Linear(3, 12),
    nn.ReLU(),
    nn.Linear(12, 64),
    nn.ReLU(),
    nn.Linear(64, 128),
    nn.ReLU(),
    nn.Linear(128, 28 * 28),
    nn.Tanh() # Output layer with Tanh activation
)

def forward(self, x):
    x = x.view(-1, 28 * 28) # Flatten the input
    encoded = self.encoder(x)
    decoded = self.decoder(encoded)
    decoded = decoded.view(-1, 1, 28, 28) # Reshape to original image dimensions
    return decoded

# Instantiate the model and move to device
model = Autoencoder().to(device)
print(model)

```

```

Autoencoder(
  (encoder): Sequential(
    (0): Linear(in_features=784, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=64, bias=True)
    (3): ReLU()
    (4): Linear(in_features=64, out_features=12, bias=True)
    (5): ReLU()
    (6): Linear(in_features=12, out_features=3, bias=True)
  )
  (decoder): Sequential(

```


[6]

✓ Os



```
# 1. Select a small sample of images from the test_loader.
# We can reuse the images and reconstructed images from the previous visualization step.
# Let's use the first 10 images from the previous batch.
sample_original_images = original_images_subset
sample_reconstructed_images = reconstructed_images_subset

# 2. Pass the selected original images through the encoder part of the trained model to obtain the compressed representation (latent vectors).
# Ensure the input is flattened as expected by the encoder
sample_original_images_flat = sample_original_images.view(-1, 28 * 28).to(device)
sample_compressed_representation = model.encoder(sample_original_images_flat)

# 3. Pass the obtained compressed representations through the decoder part of the trained model to reconstruct the images.
# This step was already done in the previous visualization subtask to get sample_reconstructed_images.
# To be explicit and follow the instructions, let's re-calculate it using the compressed representation.
sample_reconstructed_images_from_compressed = model.decoder(sample_compressed_representation).view(-1, 1, 28, 28).cpu().detach()

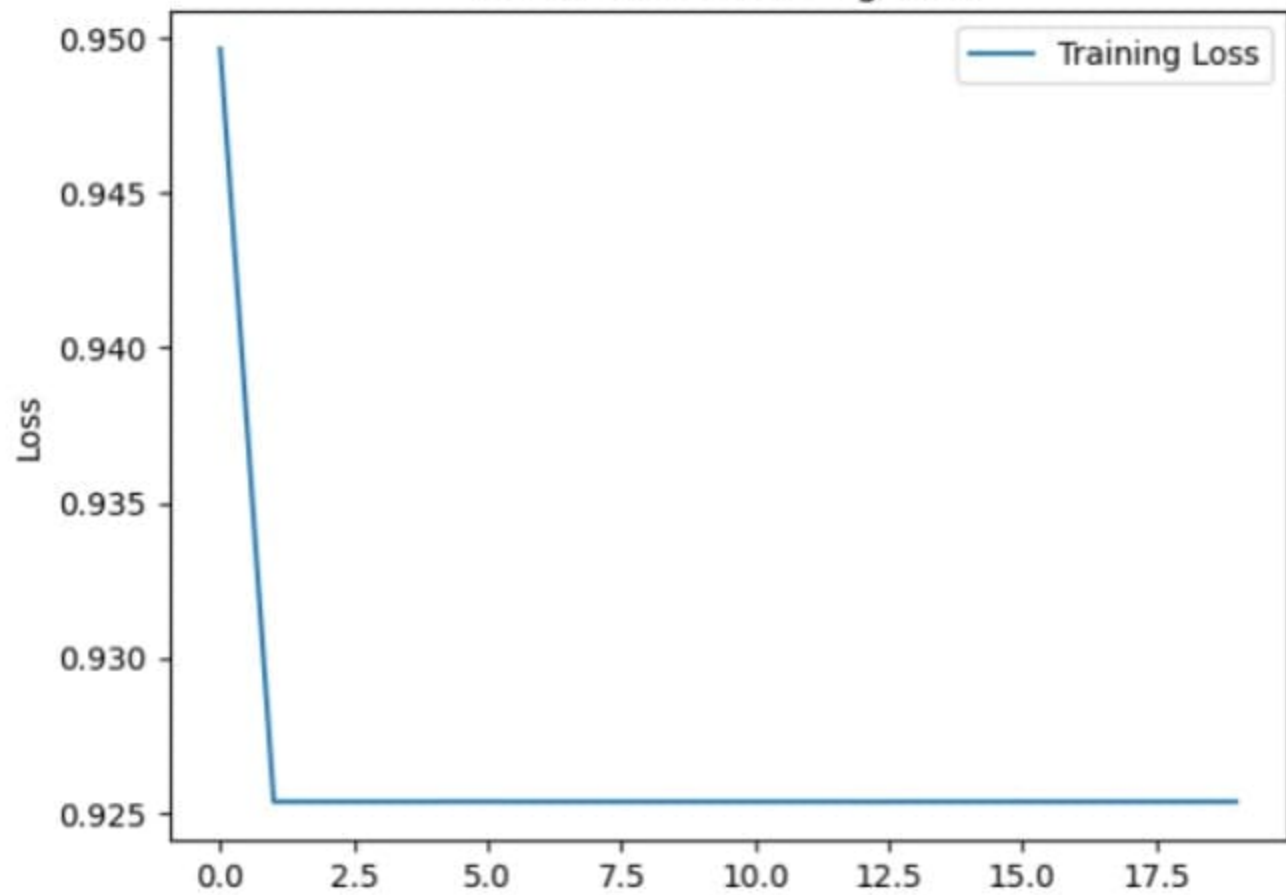
# 4. Store the original images, compressed representations, and reconstructed images for further analysis.
# The variables are already created in the previous steps:
# sample_original_images (on CPU, detached)
# sample_compressed_representation (on device, needs to be moved to CPU and detached)
# sample_reconstructed_images (on CPU, detached) - let's use the one calculated explicitly from compressed representation
sample_compressed_representation = sample_compressed_representation.cpu().detach()
sample_reconstructed_images = sample_reconstructed_images_from_compressed

print("Sample original images shape:", sample_original_images.shape)
print("Sample compressed representation shape:", sample_compressed_representation.shape)
print("Sample reconstructed images shape:", sample_reconstructed_images.shape)
```



```
Sample original images shape: torch.Size([10, 1, 28, 28])
Sample compressed representation shape: torch.Size([10, 3])
Sample reconstructed images shape: torch.Size([10, 1, 28, 28])
```

Autoencoder Training Loss





```
100%|██████████| 9.91M/9.91M [00:00<00:00, 22.1MB/s]  
100%|██████████| 28.9k/28.9k [00:00<00:00, 605kB/s]  
100%|██████████| 1.65M/1.65M [00:00<00:00, 5.52MB/s]  
100%|██████████| 4.54k/4.54k [00:00<00:00, 10.8MB/s]
```

Epoch [1/20], Loss: 0.9496

Epoch [2/20], Loss: 0.9254

Epoch [3/20], Loss: 0.9254

Epoch [4/20], Loss: 0.9254

Epoch [5/20], Loss: 0.9254

Epoch [6/20], Loss: 0.9254

Epoch [7/20], Loss: 0.9254

Epoch [8/20], Loss: 0.9254

Epoch [9/20], Loss: 0.9254

Epoch [10/20], Loss: 0.9254

Epoch [11/20], Loss: 0.9254

Epoch [12/20], Loss: 0.9254

Epoch [13/20], Loss: 0.9254

Epoch [14/20], Loss: 0.9254

Epoch [15/20], Loss: 0.9254

Epoch [16/20], Loss: 0.9254

Epoch [17/20], Loss: 0.9254

Epoch [18/20], Loss: 0.9254

Epoch [19/20], Loss: 0.9254

Epoch [20/20], Loss: 0.9254

Epoch

Original



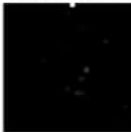
Reconstructed



Original



Reconstructed



Original



Reconstructed



Original



Reconstructed



Original



Reconstructed



Original



Reconstructed



Original



Reconstructed



Original



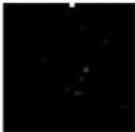
Reconstructed



Original



Reconstructed



Original



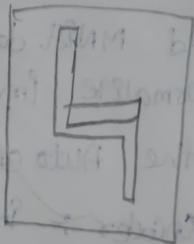
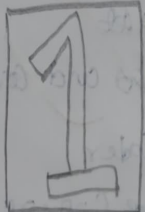
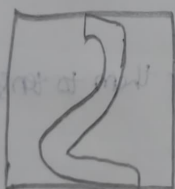
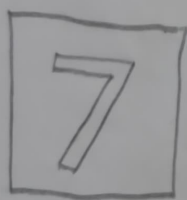
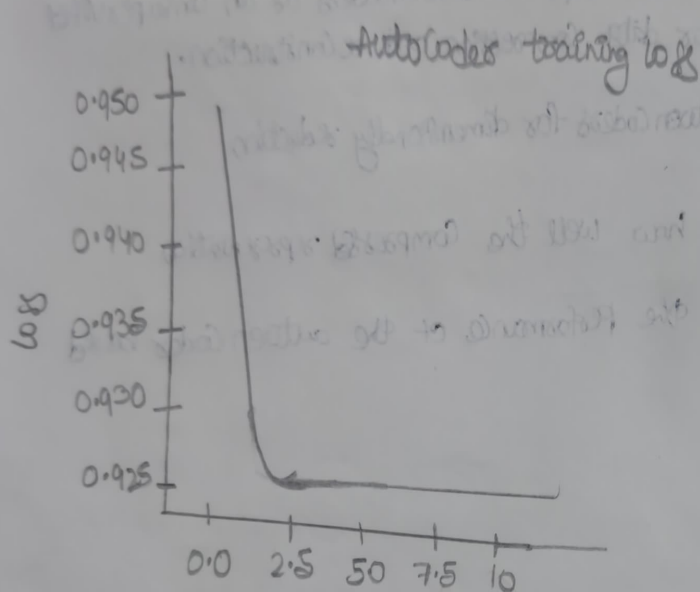
Reconstructed



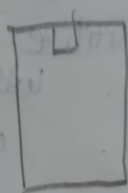
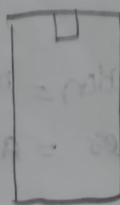
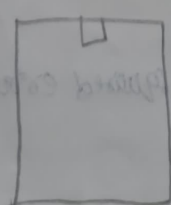
Output

Epoch : [1/10] loss : 0.9496
Epoch : [2/10] loss : 0.9254
Epoch : [3/10] loss : 0.9254
Epoch : [4/10] loss : 0.9254
Epoch : [5/10] loss : 0.9254
Epoch : [6/10] loss : 0.9254

Epoch : [7/10] loss : 0.9254
Epoch : [8/10] loss : 0.9254
Epoch : [9/10] loss : 0.9254
Epoch : [10/10] loss : 0.9254



Original Images



Reconstructed Images